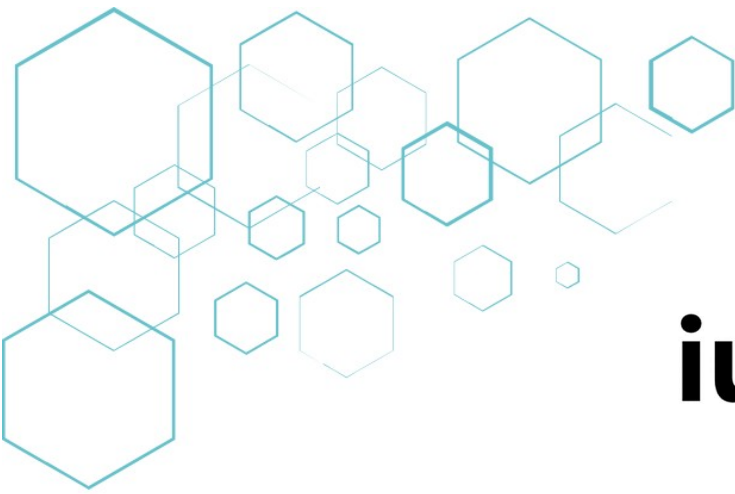




Implementierung eines Python-basierten Studien-Dashboards

Objektorientierte und funktionale Programmierung mit Python (DLBDSOOFPP01_D)

Portfolio – Abstract / Phase 3

im Studiengang
B.Sc. Cyber Security

Tutor: Dr. Aiko Walte

Eingereicht von: Evelyn Franzelin

Matrikelnummer: IU14157848

Eingereicht am: 29.05.2026

E-Mail: evelyn.franzelin@iu-study.org



Was wurde umgesetzt?

Im Rahmen der Finalisierungsphase wurde ein prototypisches Studienfortschritt-Dashboard mit Python umgesetzt. Ziel des Dashboards ist es, zentrale Informationen zum Studienverlauf übersichtlich darzustellen und dadurch eine schnelle Einschätzung des aktuellen Fortschritts zu ermöglichen. Der Prototyp visualisiert die Anzahl der abgeschlossenen Module, die erreichten ECTS-Punkte, den prozentualen Studienstand, den aktuellen Notenschnitt sowie offene Bewertungen.

Die Anwendung wurde mit Streamlit umgesetzt und verwendet eine JSON-Datei als einfache Datenquelle. Die Benutzeroberfläche wurde als browserbasierte Streamlit-Oberfläche umgesetzt. Dadurch kann der Prototyp grafisch im Browser bedient und betrachtet werden, ohne dass eine zusätzliche Desktop-Anwendung erforderlich ist. Dadurch bleibt der Prototyp schlank, nachvollziehbar und ohne zusätzliche Datenbank lauffähig. Die Studiendaten werden aus der Datei `studium.json` geladen und anschließend objektorientiert in Python-Klassen verarbeitet. Neben der Modulübersicht enthält das Dashboard auch einen Fortschrittsbalken, einen kurzen Architekturhinweis sowie einen Erfolgsmoment, der bei positiv abgeschlossenen Modulen angezeigt wird.

Wie war die Herangehensweise?

Die Umsetzung orientierte sich an der zuvor entwickelten Gesamtarchitektur. Zunächst wurden die zentralen Fachklassen definiert. Dazu gehören unter anderem `Studium`, `Modul`, `Pruefungsleistung`, `Note` und `DashboardDaten`. Ergänzend wurden Enumerationen für Studienmodell, Modulstatus, Prüfungsart und Schwierigkeit verwendet, um die Daten strukturierter und weniger fehleranfällig abzubilden.

Im nächsten Schritt wurde eine einfache Schichtenstruktur umgesetzt. Die Domain-Klassen bilden die fachlichen Objekte ab. Das Repository übernimmt das Laden und Speichern der JSON-Daten. Die Service-Klassen berechnen den Notendurchschnitt, den Studienfortschritt, die erreichten ECTS-Punkte und die offenen Bewertungen. Der Controller verbindet Repository und Services und stellt der View die fertigen Dashboard-Daten bereit. Die Streamlit-View ist ausschließlich für die Darstellung zuständig. Für die Benutzeroberfläche wurde Streamlit gewählt, da damit eine übersichtliche grafische Oberfläche mit geringem technischem Zusatzaufwand erstellt werden kann. Dies passte zum prototypischen Charakter der Aufgabenstellung, da der Schwerpunkt auf der objektorientierten Modellierung, der Datenverarbeitung und der Nachvollziehbarkeit der Architektur lag.

Diese Aufteilung wurde bewusst gewählt, um Fachlogik, Datenzugriff und Benutzeroberfläche voneinander zu trennen. Dadurch bleibt der Programmcode übersichtlich und einzelne Bestandteile können leichter erweitert oder angepasst werden.

Was lief gut?

Gut funktioniert hat vor allem die schrittweise Umsetzung der Architektur. Durch die klare Trennung in Domain, Repository, Services, Controller und View konnte jede Komponente einzeln erstellt und getestet werden. Besonders hilfreich war, zuerst mit einem kleinen Testdatensatz zu arbeiten und erst nach erfolgreicher technischer Prüfung die vollständigen Beispieldaten einzufügen.

Auch die Entscheidung für Streamlit erwies sich als sinnvoll. Die Bibliothek ermöglichte eine schnelle grafische Darstellung des Dashboards, ohne dass eine komplexe Benutzeroberfläche von Grund auf programmiert werden musste. Dadurch konnte der Schwerpunkt stärker auf der objektorientierten Struktur, der Datenverarbeitung und der Nachvollziehbarkeit der Architektur liegen.

Was war herausfordernd?

Herausfordernd war vor allem der saubere Umgang mit Dateistrukturen, Imports und JSON-Daten. Da Python-Dateien, JSON-Dateien und Projektordner korrekt zusammenspielen müssen, entstanden während der Umsetzung typische technische Fehler, etwa durch falsche Dateipfade, fehlerhafte Dateiinhalte oder Einrückungsprobleme. Diese Fehler konnten jedoch systematisch analysiert und behoben werden.

Eine weitere Herausforderung bestand darin, die Benutzeroberfläche nicht mit Fachlogik zu überladen. Gerade bei kleinen Prototypen ist es verlockend, Berechnungen direkt in der View vorzunehmen. Für eine nachvollziehbare objektorientierte Umsetzung war es jedoch wichtig, Berechnungen konsequent in Service-Klassen auszulagern und die View möglichst auf die Darstellung zu beschränken.

Welche Erkenntnisse gab es?

Eine zentrale Erkenntnis war, dass eine einfache Architektur nicht automatisch eine schlechte Architektur ist. Für den vorliegenden Prototypen war eine schlanke Lösung mit JSON-Datei, Service-Klassen und Streamlit ausreichend und passend. Eine Datenbank hätte die Anwendung zwar technisch erweitert, aber für den aktuellen Umfang auch unnötig komplex gemacht.

Außerdem wurde deutlich, wie wichtig kleine Zwischentests sind. Durch separate Testdateien für Domain-Klassen, Repository, Services und Controller konnte früh geprüft werden, ob die einzelnen Bestandteile korrekt funktionieren. Dadurch ließen sich Fehler besser eingrenzen und beheben.

Worauf bin ich besonders stolz?

Besonders zufrieden bin ich damit, dass aus der vorherigen Planung ein tatsächlich lauffähiger Prototyp entstanden ist. Das Dashboard zeigt nicht nur statische Informationen an, sondern berechnet

zentrale Kennzahlen aus den hinterlegten Studiendaten. Dazu gehören der Studienstand, die erreichten ECTS-Punkte, der Notenschnitt und offene Bewertungen.

Stolz bin ich außerdem auf die klare Struktur des Projekts. Die Anwendung ist nicht nur optisch verständlich, sondern auch im Code nachvollziehbar aufgebaut. Der Architekturhinweis im Dashboard macht zusätzlich sichtbar, wie die einzelnen Schichten zusammenarbeiten.

Wie können die Ergebnisse weiterverwendet werden?

Der Prototyp kann als Grundlage für eine erweiterte Studienverwaltungsanwendung dienen. In einer späteren Version könnten alle Module des Studiengangs hinterlegt, Noten komfortabel über die Benutzeroberfläche gepflegt und weitere Kennzahlen ergänzt werden. Denkbar wären außerdem Filterfunktionen, Diagramme, Exportmöglichkeiten oder eine automatische Berechnung erwarteter Bewertungstermine.

Auch die Datenhaltung könnte erweitert werden. Für den Prototypen reicht eine JSON-Datei aus. Bei einer größeren Anwendung wäre jedoch eine Datenbank sinnvoll, um größere Datenmengen, mehrere Nutzerinnen und Nutzer oder regelmäßige Änderungen robuster zu verwalten. Die vorhandene Schichtenstruktur bietet dafür eine gute Grundlage, weil das Repository ausgetauscht werden könnte, ohne die gesamte Anwendung neu aufzubauen.

Hinweis zur KI-Nutzung: Ein KI-Chatbot wurde punktuell zur sprachlichen Überarbeitung und zur Prüfung der Verständlichkeit einzelner Textstellen genutzt. Die Konzeption, fachliche Bewertung und finale Ausarbeitung erfolgten eigenständig.